

Preprocessing Pipeline

Step 2 — Cleaning the Target Column (Price)

The `Price` column was stored as a string with currency symbols and commas (e.g. "\$12,500"), making it impossible to use numerically. I used `.replace` with a regex pattern `[\$,,]` to strip those characters and then cast the column to `float`. I checked the skewness after conversion to decide whether outlier treatment was needed — it was, which led directly to Step 6.

Step 3 — Fixing Category Errors Before Imputation

I fixed the `Location` column before imputation deliberately, not after. If I had imputed first, the type "Subrb" and the unknown "??" would have been counted as valid categories and potentially used as the mode — producing a meaningless fill value. By correcting and nullifying them first, the mode calculated in Step 4 is based only on genuinely valid entries (`City`, `Suburb`, `Rural`).

Step 4 — Imputing Missing Values

Each column received a different imputation strategy based on its data type and distribution:

Column	Method	Justification
<code>`Odometer_km`</code>	Median	Continuous numeric column. Median is more robust than mean when the distribution is skewed or has outliers — which odometer readings typically are.
<code>`Doors`</code>	Mode	Discrete categorical-style integer. The most common door count (likely 4) is the most sensible default — a mean would produce a non-integer like 3.8, which is meaningless.
<code>`Accidents`</code>	Mode	Binary or low-cardinality column. Mode gives the most typical observed value without inventing fractional accident counts.
<code>`Location`</code>	Mode	Categorical text. Mean is undefined; mode fills with the most represented location category, which is the least biased choice.

Step 5 — Removing Duplicates

Duplicate rows can cause a model to overfit to specific repeated records, effectively giving those examples more weight during training than they deserve. I used `drop_duplicates()` and saved the result back to `df` to ensure the cleaning was actually applied (not just computed and discarded).

Step 6 — Outlier Handling (IQR Capping)

I chose **IQR capping** rather than deletion for two reasons:

1. **No data loss** — deleting outliers reduces the dataset size; capping retains every row while limiting the influence of extreme values.
2. **Preserves distribution shape** — the values are clipped to the IQR-derived bounds ($Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$), so the rest of the distribution is untouched.

I applied this to both `Price` (the target, which showed high skewness) and `Odometer_km` (which similarly had extreme high-end values from old high-mileage vehicles).

Step 7 — One-Hot Encoding Location

`Location` is a nominal categorical variable with no inherent order (`City`, `Suburb`, `Rural` are not ranked), so ordinal encoding would be inappropriate. I used `pd.get_dummies` with `drop_first=False` to create a separate binary column for each category. I kept all three columns rather than dropping one because the dropped column's information is still needed for the engineered `Is_Urban` feature in Step 8, and to keep the encoding transparent and interpretable.

Step 8 — Feature Engineering

I added three new features, each designed to carry information not already explicitly present:

Feature	Formula	Reasoning
<code>`Car_Age`</code>	<code>`2026 - Year`</code>	Raw year is hard to interpret; age is directly meaningful for predicting price depreciation.
<code>`Km_per_year`</code>	<code>`Odometer_km ÷ Car_Age`</code>	Separates high-mileage-because-old from high-mileage-because-heavily-used. A 10-year-old car with 200,000 km is very different from a 2-year-old car with the same reading. Division by zero (brand-new cars) is handled by replacing <code>Car_Age = 0</code> with <code>NaN</code> before dividing.
<code>`Is_Urban`</code>	<code>`Location_Suburb == 1`</code>	Collapses the <code>Location</code> dummies into a single binary: urban (<code>Suburb=1</code>) vs non-urban (<code>City</code> or <code>Rural=0</code>), which may have cleaner predictive signal for price than three separate dummies.

`LogPrice = log(Price + 1)` was added as an **alternative regression target**, not a feature. It compresses the right-skewed price distribution, which can improve model performance for algorithms sensitive to scale. The `+1` prevents `log(0)` errors.

Step 9 — Feature Scaling

I applied `StandardScaler` (zero mean, unit variance) only to continuous numeric columns. I deliberately excluded:

3. `Price` and `LogPrice` — these are target variables, not inputs. Scaling them would distort the regression objective.
4. `Location_*` dummy columns and `Is_Urban` — binary 0/1 columns already have a bounded, interpretable scale. Standardizing them would make the output harder to interpret without adding any benefit for most algorithms.

Scaling is necessary because algorithms like linear regression and SVM are sensitive to feature magnitude — an unscaled `Odometer_km` (range: 0–200,000) would numerically dominate `Doors` (range: 2–5) and produce biased coefficients.